

Les Réseaux de Neurones Artificiels

Partie 2 : Réseaux profonds et autres architectures

Explication du cours — diapositives 96 à la fin (153)

Notes de lecture, définitions et intuitions

Ce document poursuit l'explication du cours là où le précédent s'était arrêté. On y quitte le perceptron multi-couches « générique » pour entrer dans le cœur du deep learning moderne : les réseaux convolutionnels (CNN), la manière dont ils reconnaissent des images, comment on entraîne un réseau quand on manque de données étiquetées, comment on stabilise l'apprentissage de réseaux très profonds, et on termine par un résultat surprenant qui remet en question notre compréhension de la généralisation, avant un tour d'horizon d'architectures alternatives (reservoir computing, réseaux impulsionnels).

1. Vers les réseaux profonds : l'ancêtre des réseaux convolutionnels

Le problème historique : lire des codes postaux manuscrits

Le point de départ (Le Cun et al., 1989, AT&T Bell Labs) est un problème très concret : reconnaître automatiquement des chiffres manuscrits sur des enveloppes postales.

- $\approx 10\,000$ exemples de chiffres manuscrits, segmentés et redimensionnés sur une matrice de 16×16 pixels.
- Résultat : 99 % de reconnaissance correcte sur l'ensemble d'apprentissage, avec 9 % de cas « rejetés » (renvoyés à un opérateur humain plutôt que risquer une erreur).

Pour résoudre ce problème, deux idées nouvelles apparaissent par rapport au perceptron multi-couches « classique » vu jusqu'ici :

- **Le partage de poids (weight sharing)** : au lieu de donner à chaque neurone son propre jeu de poids indépendant, on force plusieurs neurones à utiliser exactement les mêmes poids. L'idée : un « détecteur de trait » utile (par exemple un bord orienté à 45°) doit pouvoir être appliqué identiquement à n'importe quel endroit de l'image — inutile de réapprendre le même détecteur à chaque position.
- **L'optimal brain damage** : une technique d'élagage qui supprime après coup les connexions les moins utiles du réseau, pour le simplifier sans trop dégrader ses performances.

La convolution : définition et intuition

Une convolution consiste à faire glisser un petit filtre (une petite grille de poids, par exemple 3×3 ou 11×11) sur toute l'image, et à calculer à chaque position la somme pondérée des pixels recouverts par le filtre. C'est exactement le même calcul qu'un neurone classique (somme pondérée + activation), sauf que :

- le filtre est petit et ne regarde qu'une petite région locale de l'image à la fois (une « fenêtre » ou receptive field),
- le même filtre (les mêmes poids) est réutilisé à toutes les positions de l'image — c'est le partage de poids mentionné plus haut.

Exemple de calcul (glisser un filtre 3×3 sur une portion de l'image) :

$$(4 \times 0) + (0 \times 0) + (0 \times 0) + (0 \times 0) + (0 \times 1) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (-4 \times 2) = -8$$

Chaque valeur ainsi calculée devient un pixel de la « carte de traits » (feature map) en sortie. Le résultat : au lieu d'une image brute, on obtient une nouvelle image qui indique, à chaque position, si le motif recherché par le filtre (un bord, une courbure, une texture...) est présent ou non.

Pourquoi c'est puissant : un même détecteur de bord entraîné dans un coin de l'image sert automatiquement partout ailleurs. Cela réduit énormément le nombre de paramètres à apprendre par rapport à un réseau totalement connecté, et rend le réseau insensible à la position exacte du motif dans l'image (invariance par translation).

Le max-pooling : réduire la résolution intelligemment

Après une convolution, on applique souvent une opération de max-pooling : on découpe la carte de traits en petits blocs (par exemple 2×2) et on ne garde, pour chaque bloc, que la valeur maximale.

$[[1,3,2,1],[4,2,0,3],[5,1,2,2],[0,3,1,0]] \xrightarrow{\text{max-pooling } 2 \times 2} [[4,3],[5,2]]$

L'intérêt est double : cela réduit la taille des données à traiter par les couches suivantes (donc le coût de calcul), et cela rend la reconnaissance plus robuste à de petits décalages ou déformations du motif recherché — seule la présence du signal le plus fort dans une zone compte, pas sa position exacte au pixel près.

L'architecture générale d'un réseau convolutionnel

En combinant ces deux briques, un CNN typique alterne plusieurs fois les étapes suivantes avant de terminer par un classifieur classique :

1. **Convolution** : des filtres apprennent à détecter des motifs locaux (bords, textures...).
2. **Pooling** : on réduit la résolution des cartes de traits obtenues (par exemple en moyennant ou en prenant le maximum de pixels contigus).
3. **Nouvelle convolution** : de nouveaux filtres sont appris sur ces cartes déjà simplifiées, ce qui leur permet de détecter des motifs de plus en plus complexes et abstraits (un œil, une roue, un visage...).
4. **Classifieur** : les toutes dernières couches sont entièrement connectées (un MLP classique) et apprennent à prédire la classe finale à partir de ces descripteurs générés automatiquement par les couches précédentes.

Idée clé à retenir : un CNN n'a pas besoin qu'on lui dise à l'avance quels motifs chercher dans l'image — les filtres de convolution sont eux-mêmes appris par rétropropagation, exactement comme les poids d'un perceptron multi-couches classique. La seule différence est la contrainte de structure (poids partagés, connexions locales) imposée à l'architecture.

2. Le rôle du choix de représentation des entrées

Le cours insiste sur un point méthodologique important, valable pour tout apprentissage automatique, pas seulement les réseaux de neurones :

Principe : « L'apprentissage devient facile quand on sait déjà quoi chercher. » Un bon choix de représentation des données peut rendre une tâche d'apprentissage presque triviale — mais comment savoir a priori quelle est la bonne représentation ?

Deux questions se posent systématiquement avant d'attaquer un problème :

- De quel type de tâche s'agit-il ? (reconnaissance de formes, de caractères, de locuteurs...)
- Comment coder les exemples en entrée du réseau ? (pixels bruts, descripteurs faits main, autre encodage...)

L'intérêt des réseaux profonds (et en particulier des CNN) est justement de réduire la dépendance à ce choix : au lieu de fabriquer des descripteurs à la main, on laisse le réseau les apprendre lui-même directement à partir des pixels bruts, couche après couche.

3. Quelques architectures emblématiques

LeNet-5

LeNet-5 est l'architecture de référence héritée des travaux sur les codes postaux, formalisée pour la reconnaissance de caractères. Elle alterne couches de convolution/activation, couches d'agrégation (pooling), puis couches complètement connectées, terminées par une couche de sortie de type softmax (une probabilité par classe).

La compétition ImageNet et AlexNet

ImageNet est une compétition de reconnaissance d'images qui a joué un rôle décisif dans l'essor du deep learning :

- Plus de 15 millions d'images haute résolution étiquetées.
- Environ 22 000 catégories différentes.
- Images récoltées sur le Web et étiquetées via une plateforme de micro-travail (Amazon Mechanical Turk).

AlexNet (Krizhevsky, Sutskever & Hinton, 2012) est le CNN qui a marqué un tournant en dominant cette compétition. Le cours donne un exemple concret pour montrer à quel point le nombre de paramètres explose vite, même avec un réseau qui reste peu profond :

- Entrée : images de $227 \times 227 \times 3$ (3 canaux de couleur).
- Première couche de convolution : filtres de taille 11×11 , avec un décalage (stride) de 4 entre chaque application du filtre, ce qui produit une sortie de $55 \times 55 \times 96$ neurones.
- Chaque neurone de cette seule couche a $11 \times 11 \times 3 = 363$ poids + 1 biais → cela représente déjà 105 705 600 paramètres à ajuster, rien que pour la toute première couche.

À retenir : cet exemple illustre pourquoi l'entraînement de réseaux profonds nécessite beaucoup de données, beaucoup de puissance de calcul, et des techniques particulières pour éviter le sur-apprentissage (voir la section sur la régularisation plus loin).

U-Net : la segmentation d'images

U-Net est une architecture conçue pour la segmentation d'image : au lieu de prédire une seule classe pour toute l'image (« c'est un chat »), on veut prédire une classe pour chaque pixel (« ce pixel appartient au chat, celui-ci au fond »).

Son architecture, en forme de U, est entièrement convolutive et symétrique :

- Dans la première moitié (descente), la taille spatiale de l'image diminue progressivement tandis que le nombre de filtres (donc la richesse de la représentation) augmente — le réseau « comprend » de mieux en mieux le contenu, mais perd en résolution spatiale.
- Dans la seconde moitié (montée), le processus s'inverse : on reconstruit progressivement une carte de la même résolution que l'image d'origine, mais où chaque pixel porte une prédiction de classe. Des connexions directes (copy and crop) relient les couches symétriques pour ne pas perdre les détails fins.

GoogleNet et les très grands réseaux

GoogleNet illustre le passage à l'échelle : c'est un assemblage (« mécano ») de blocs convolutionnels combinés de façon plus sophistiquée qu'une simple pile séquentielle. Le cours cite un exemple de système développé par Google et Stanford pour la reconnaissance de visages sous des conditions de lumière et d'angle très variables, entraîné de façon non supervisée avec :

- 9 couches, environ 10^9 connexions,
- 10 millions d'images,
- 3 jours de calcul sur 16 000 processeurs,
- pour une amélioration de 70 % par rapport à l'état de l'art de l'époque.

4. Que représentent réellement les neurones cachés ?

Une symétrie féconde entre poids et exemples

Dans la descente de gradient par rétropropagation, il existe une forme de symétrie mathématique entre le rôle joué par les poids des connexions et celui joué par les exemples d'entrée. Une conséquence pratique intéressante : on peut « retourner » le problème et optimiser une image d'entrée (plutôt que les poids) pour qu'elle active au maximum un neurone caché donné.

Utilité : cette technique (feature visualization) permet de visualiser concrètement ce à quoi « répond » chaque neurone caché — un moyen de percer, au moins partiellement, l'opacité des réseaux de neurones évoquée en introduction.

L'apprentissage de représentations hiérarchiques

C'est sans doute l'observation la plus marquante sur le fonctionnement interne des CNN entraînés sur des images : les différentes couches successives apprennent spontanément une hiérarchie de représentations, de plus en plus abstraites :

- Couche 1 (proche de l'entrée) : détecte des motifs très simples et locaux — des bords orientés, des contrastes.
- Couche 2 : combine ces bords pour détecter des parties d'objets — un œil, un nez, une oreille.
- Couche 3 (proche de la sortie) : combine ces parties pour reconnaître des objets entiers — un visage complet.

Le même phénomène s'observe pour d'autres types de signaux (la parole, le texte) : les couches basses captent des régularités locales, les couches hautes des représentations linguistiques ou sémantiques de haut niveau. C'est cette capacité à apprendre automatiquement des abstractions successives, sans qu'on les ait programmées à la main, qui caractérise le deep learning.

5. Que faire quand on manque de données étiquetées ?

Les réseaux profonds ont besoin d'énormément d'exemples étiquetés pour bien apprendre (voir l'exemple AlexNet avec ses centaines de millions de paramètres). Or étiqueter des données à la main coûte cher et prend du temps. Plusieurs stratégies existent pour contourner ce problème.

L'augmentation de données (data augmentation)

L'idée est de générer artificiellement de nouveaux exemples d'entraînement à partir des exemples étiquetés déjà disponibles :

- **Transformations simples** : bruitage, flou, changements de contraste ou de luminosité, rotation, dilatation, translation... On modifie légèrement une image existante tout en sachant que son étiquette reste inchangée (un chat légèrement pivoté reste un chat).
- **Génération par réseaux antagonistes (GANs, Generative Adversarial Networks)** : des réseaux génératifs plus sophistiqués créent des exemples entièrement synthétiques. Le cours prévient que cette approche, bien que puissante, n'est « pas évidente du tout à utiliser » en pratique.

L'auto-encodage (autoencoder)

Une idée ancienne mais toujours centrale : l'auto-association. Un réseau est composé de deux parties :

- **un encodeur**, qui compresse l'entrée x en une représentation interne réduite (l'embedding),
- **un décodeur**, qui tente de reconstruire l'entrée d'origine à partir de cette seule représentation compressée.

Le réseau est entraîné à reproduire sa propre entrée en sortie ($x \rightarrow \hat{x} \approx x$), à travers un « goulot d'étranglement » de dimension réduite. Pour y parvenir, l'encodeur est contraint d'apprendre une représentation compacte mais informative des données — l'embedding — qui capture leur structure essentielle (on observe par exemple que les chiffres manuscrits similaires se regroupent naturellement dans cet espace réduit).

Point subtil : c'est un apprentissage supervisé dans sa mécanique (il y a une cible à atteindre, x lui-même) mais sans aucune étiquette humaine requise — d'où son intérêt quand on manque de données annotées.

L'auto-apprentissage (self-supervised learning)

Dans la même veine, l'apprentissage auto-supervisé est une forme d'apprentissage non supervisée (aucune étiquette humaine nécessaire) qui se ramène pourtant, techniquement, à un apprentissage supervisé : on fabrique automatiquement des paires (entrée, étiquette) à partir des données brutes elles-mêmes, en imaginant une tâche annexe (pretext task) dont la solution nécessite de comprendre la structure des données.

Exemple donné dans le cours : entraîner un réseau à prédire la position relative de deux petits morceaux (patches) extraits d'une même image (par exemple : « ce morceau est-il en haut à droite ou en bas à gauche de cet autre morceau ? »). Pour réussir cette tâche, le réseau est obligé d'apprendre des connaissances relationnelles sur ce qui constitue une image cohérente (formes, textures, contexte) — des connaissances ensuite réutilisables pour la vraie tâche qui nous intéresse. La même idée s'applique à des séquences (déterminer si une séquence est « valide » ou non, prédire l'élément suivant, etc.).

6. Stabiliser et régulariser l'apprentissage des réseaux profonds

Plus un réseau est profond, plus son entraînement devient délicat (instabilité numérique, sur-apprentissage accru vu le grand nombre de paramètres). Deux familles de techniques très utilisées en pratique :

Les couches de normalisation

Ces couches modifient la sortie des neurones en fonction de l'activité des autres neurones, pour stabiliser les distributions de valeurs qui circulent dans le réseau au fil de l'entraînement :

- **Batch normalization** : on normalise chaque descripteur (chaque dimension) indépendamment, en utilisant la moyenne et l'écart-type calculés sur l'ensemble des exemples du lot (batch) d'entraînement courant.
- **Layer normalization** : à l'inverse, on normalise chaque exemple indépendamment, en utilisant la moyenne et l'écart-type calculés sur l'ensemble des descripteurs de cet exemple.
- **L2-norm** : une autre variante qui projette les sorties sur une sphère de norme constante (déjà rencontrée comme exemple de transformation non linéaire).

Dans tous les cas, l'objectif est le même : empêcher les valeurs qui circulent dans le réseau de devenir trop grandes, trop petites, ou trop disparates d'une couche à l'autre, ce qui accélère et stabilise la convergence de la descente de gradient.

Le Dropout

Le dropout est une technique de régularisation très simple dans son principe : pendant l'entraînement, à chaque itération, on désactive aléatoirement une fraction des neurones (et de leurs connexions), de façon temporaire.

Intuition : en empêchant les neurones de trop « compter » les uns sur les autres (co-adaptation), on force le réseau à apprendre des représentations plus redondantes et plus robustes, qui ne dépendent pas de la présence d'un neurone particulier. C'est un peu comme s'entraîner en équipe en sachant que n'importe quel coéquipier peut être absent le jour du match : chacun doit être capable de compenser. Au moment de l'utilisation finale du réseau (test), tous les neurones sont réactivés.

7. Un résultat troublant : faut-il repenser la généralisation ?

Le cours présente une expérience célèbre (Zhang, Bengio, Hardt, Recht & Vinyals, ICLR 2017 — « Understanding deep learning requires rethinking generalization ») qui bouscule l'intuition du compromis biais-variance présentée dans la première partie du cours.

L'expérience

Sur le jeu de données CIFAR-10 (60 000 images en couleur de 32×32 pixels, réparties en 10 classes), les auteurs entraînent AlexNet (plus d'un million de paramètres) et deux autres architectures, dans trois conditions :

1. **Données originales, non modifiées** : précision d'entraînement = 100 %, précision de test = 89 %.
Résultat attendu, rien d'étonnant ici.

2. **Étiquettes aléatoires** : on remplace les vraies étiquettes par des étiquettes tirées au hasard, complètement déconnectées du contenu de l'image. Résultat : précision d'entraînement = 100 % quand même (!), et une vitesse de convergence à peine plus lente.
3. **Pixels aléatoires** : on remplace même le contenu des images par du bruit aléatoire pur. Résultat : précision d'entraînement = 100 % toujours (!!), avec une vitesse de convergence à nouveau comparable.

Ce qui est troublant : un réseau de neurones profond peut apprendre par cœur n'importe quel ensemble d'exemples, même totalement dénué de structure — sa capacité mémorielle semble quasiment illimitée. Ce comportement contredit l'intuition classique du compromis biais-variance, selon laquelle un modèle capable de « tout » mémoriser devrait systématiquement sur-apprendre et donc mal généraliser.

La question ouverte que pose cette expérience : si un réseau profond est de toute façon capable de mémoriser n'importe quoi, pourquoi généralise-t-il si bien en pratique sur de vraies données (89 % de précision de test dans le premier cas) plutôt que de se contenter de mémoriser le bruit ? C'est une question de recherche encore activement débattue aujourd'hui : la théorie classique de l'apprentissage statistique (biais-variance, dimension de Vapnik-Chervonenkis) ne suffit pas, à elle seule, à expliquer pourquoi le deep learning fonctionne aussi bien qu'il le fait.

8. Autres architectures : au-delà du réseau « feedforward »

Le réservoir computing

Une idée assez contre-intuitive : utiliser un grand réseau récurrent (avec des boucles internes, donc une forme de mémoire) — mais sans jamais entraîner ses poids internes ! Seule la toute dernière couche de sortie (readout) est entraînée, par une simple régression linéaire.

- **Le « réservoir »** est un grand réseau récurrent aux poids fixés une fois pour toutes, aléatoirement, dès le départ. Une entrée qui le traverse y génère des dynamiques internes riches et complexes (des « échos » de l'entrée qui se propagent et se combinent dans le temps).
- **Seule la couche de lecture** en sortie est entraînée, pour apprendre à combiner ces états internes du réservoir afin de produire la sortie désirée.

Cette approche réintroduit une notion de dynamique temporelle (utile par exemple pour prédire des séries temporelles) tout en évitant les difficultés d'entraînement des réseaux récurrents profonds par rétropropagation (qui sont notoirement instables sur de longues séquences).

Les réseaux à neurones impulsionnels (spiking neural networks)

Cette famille de modèles cherche à se rapprocher davantage du fonctionnement réel des neurones biologiques évoqué tout au début du cours. Contrairement aux neurones formels vus jusqu'ici (qui produisent une valeur continue à chaque instant), un neurone impulsif communique par de brèves impulsions (spikes) discrètes dans le temps :

- Chaque impulsion reçue d'un neurone pré-synaptique génère un petit potentiel (excitateur ou inhibiteur selon le signe du poids de la connexion).
- Ces potentiels s'additionnent au cours du temps dans le neurone receveur.
- Lorsque le potentiel cumulé dépasse un certain seuil, le neurone émet à son tour une impulsion vers les neurones suivants, puis son potentiel retombe.

C'est un modèle plus fidèle à la biologie (voir l'introduction du cours) mais aussi beaucoup plus difficile à entraîner avec les outils classiques de descente de gradient, puisque l'émission d'une impulsion est un événement discret, non dérivable au sens usuel.

9. À retenir (bilan général du cours)

1. **Une méthode d'induction supervisée très versatile** : un très grand champ d'application, une famille de modèles très souple — mais certainement pas un modèle biologique fidèle, malgré l'inspiration initiale.
2. **Un apprentissage qui demande du soin** : le choix de l'architecture reste largement empirique, l'apprentissage est lent et sujet aux minima locaux.
3. **Une « opacité » persistante** : il reste difficile d'interpréter un réseau entraîné, et difficile d'y injecter de la connaissance a priori — deux limites qui rejoignent directement les « défauts » évoqués en tout début de cours.

Références bibliographiques

Auteurs	Titre	Éditeur
Barra, Cornuéjols & Miclet	<i>Apprentissage artificiel. Concepts et algorithmes. De Bayes et Hume au Deep Learning</i>	Eyrolles, 2021
Bishop & Bishop	<i>Deep Learning: Foundations and Concepts</i>	Springer, 2023
Haykin	<i>Neural Networks. A comprehensive foundation</i>	Prentice Hall, 1999
Raschka, Liu & Mirjalili	<i>Machine Learning with PyTorch and Scikit-Learn</i>	Packt, 2022
Torralba, Isola & Freeman	<i>Foundations of Computer Vision</i>	MIT Press, 2024

Document préparé à partir du cours « Les réseaux de neurones artificiels » (A. Cornuéjols, AgroParisTech – INRAE MIA Paris-Saclay).